

Distributed Systems

Consensus



CUHK

Consensus Problem

- N processes $\{0, 1, 2, \dots, n-1\}$
- **Each process proposes its own initial value** in an agreed domain
- **Crash Fault tolerant**
- In the end, every process agrees upon an irrevocable final decision value
 - The losers' proposals are discarded
- **Agreement:** that value of a nonfaulty process must be identical to other nonfaulty processes
- **Termination:** every nonfaulty process must reach a decision of its final value
 - Liveness
- **Validity:** just a sanity check. If every nonfaulty process begins with the same initial value v , then their final decision must be v (i.e., it's absurd to find another value to agree on that reflects nobody's initial choice)

Specialized Consensus Problems

- Leader Election

- Final decision value – every process agrees on the same leader id

- Distributed Transaction

- Sharding (Partitioning) but not Replication
- Transfer \$100 from my CA branch to NY branch
- Atomicity: Commit or Abort (all or nothing)
- Consensus on what?
 - Value = {commit, abort}
 - CA commit, NY commit
 - CA abort, NY abort
 - All or nothing

- Agreement Problem

- Single Proposal (everybody agrees to go McDonald?) yes / no only

- RSM

- A client submits 'put(x=7)', all replicas agree/disagree about it

Hardness of Consensus Problem

- FLP Theorem [Journal of ACM 85]
 - “Impossible to have a **deterministic** protocol that solves **consensus** in an **asynchronous** system with ≥ 1 process may crash-fail”
 - Corollary: “no consensus protocol can guarantee termination (liveness) in practice”
 - It also extends to the hardness of complete vs. accurate in failure detection
 - Baseline algorithm when no faulty processes
 - Everybody sends its own v to all others
 - Let the collection be V (and currently we have no malicious case yet)
 - Everyone gets the same V
 - Everyone applies a function f on V
 - Example f can be the majority (as long as it is deterministic)
 - Agreement done!



Fischer



Lynch



Paterson

Intuition of FLP theorem

- At first glance it might be counter-intuitive

- N processes and take majority:

- 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1

- So if  is gone, nothing hurt.

- But FLP is a **theorem**, talking about the corner/worst case:

- 0 0 0 1 1 1 1

- If the **faulty process** is gone, then it hurts!

Hardness of Consensus Problem

- Asynchronous System

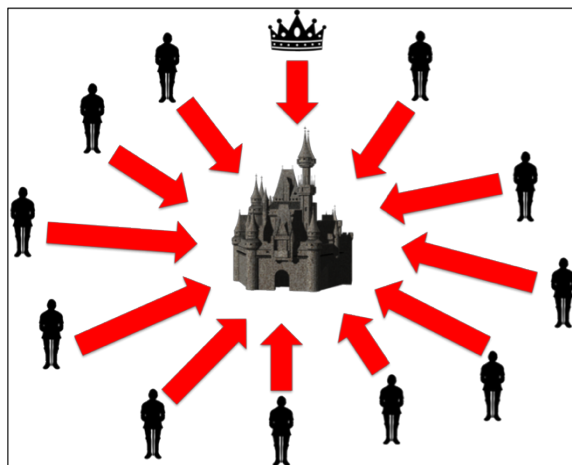
- Impossibility of distributed consensus with one faulty process
- If crash-fault is impossible, dealing with **Byzantine faults** would be even more impossible

- Synchronous System

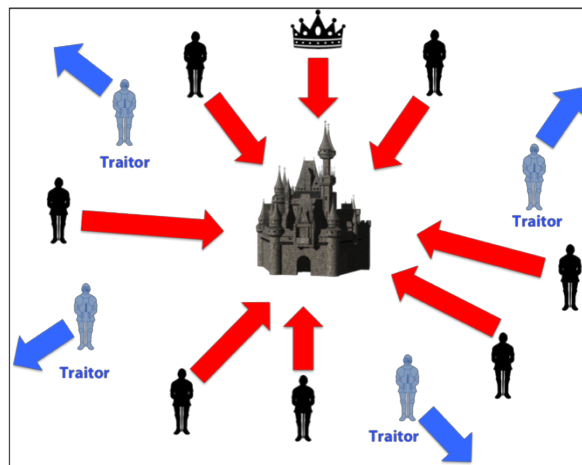
- Byzantine fault
 - A.k.a. Byzantine Generals Problem
 - Impossible when # of processes $n \leq 3$ and malicious process $f = 1$
 - In other words, **possible** when $n > 3$ when $f = 1$
 - Cast it as n generals, find an agreed time to attack all together or else leads to defeat

Byzantine Generals Problem (remember: sync mode)

- Traitors might:
 - Not responding
 - Send conflicting messages to hinder the agreement
 - No one know who the traitors are



Coordinated Attack Leading to Victory



Uncoordinated Attack Leading to Defeat

Byzantine Generals Problem

- N generals
 - Any general can send message {attack 1 or retreat 0} //hence, is a consensus problem
- When publishing, the general is playing the role of the unique/one commander
- When receiving, the general is playing the role of a lieutenant
- Consistency Requirements (named as “interactive consistency”):
 - IC1: All loyal receivers (lieutenants) receive the same message from the same sender
 - →it doesn't rule out the awkward case of the commander sends out “Attack” but all loyal receivers receive the same “Retreat” message
 - IC2: If a sender (commander) is loyal, every loyal receiver receives his order

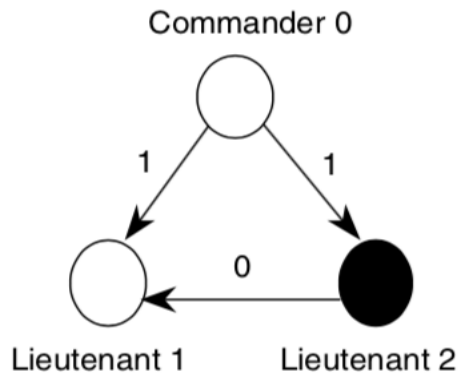
BGP settings



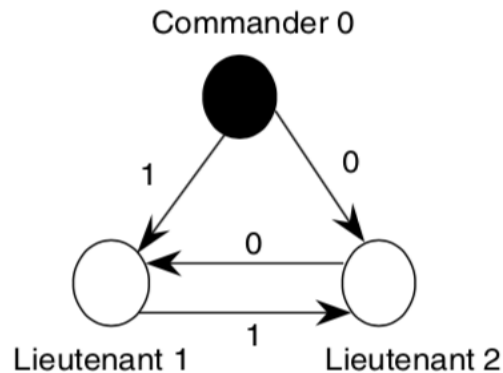
- Communication channel is “Oral Message”
 - Messages are not corrupted in transit
 - (loyal messenger; though the message itself can be wrong because of the bad sender)
 - Messages can be lost, but absence of message can be detected (=synchrony!)
 - When a message is received (or its lost is detected)
 - the receiver knows who is the sender of that message

Byzantine Generals Problem

- Using oral message, impossible when
 - # of processes $n \leq 3$ and malicious process $f = 1$



(a)



(b)

- Generalizable to: using oral message, impossible when
 - # of processes $n \leq 3f$ and f traitors ($f > 0$)
 - Why highlight oral message?
 - If the commander can write and sign the message, case (a) is gone

Lamport et al “Oral Message” algorithm

- As a lieutenant, if I am smart, before I attack, I will check if other lieutenants (receivers) receive the same messages as mine
- So, everyone forwards its received messages to the others
- Hence, there are two kinds of messages:
 - Direct message: received from the commander directly
 - Indirect message: 2nd hand message received from the other lieutenants about the commander's messages

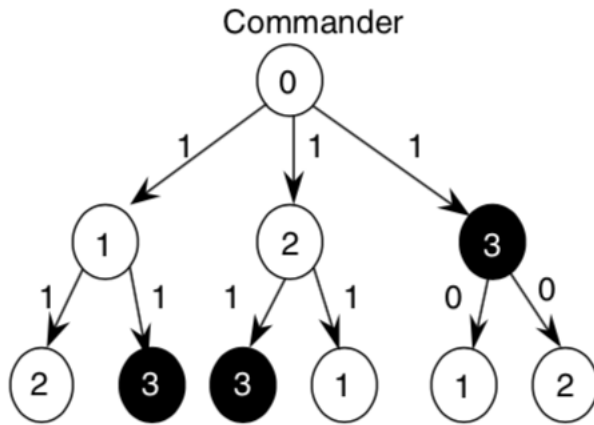
Lamport et al 1982 OM algorithm

- A recursive algorithm:
 - f is set as the number of traitors
 - The higher f , more recursive levels are needed
- $OM(0, v)$ //base case
 - Commander i sends out v (0 or 1) to every lieutenant j ($j \neq i$)
 - Each lieutenant j accepts the value from i
- $OM(f, v)$ //3 phases, for $f \geq 1$
 - 1) The commander i sends out a value v (0 or 1) to every lieutenant j ($j \neq i$)
 - 2) Each lieutenant j who receives v
 - Recursive call $OM(f-1, v)$ as a new commander to forward v to the other lieutenants (but not i)
 - //All (honest) lieutenants would do this, hence outvotes the malicious ones
 - 3) Each lieutenant j , receives $(n-1)$ values in total:
 - 1 value directly from commander i 's $OM(f, v)$ //from step 1
 - $n-2$ indirect values from $n-2$ lieutenants j . $OM(f-1, v)$ //from step 2
 - Each lieutenant uses the majority of the $(n-1)$ values
 - If a value is not received, assume that as 'retreat'

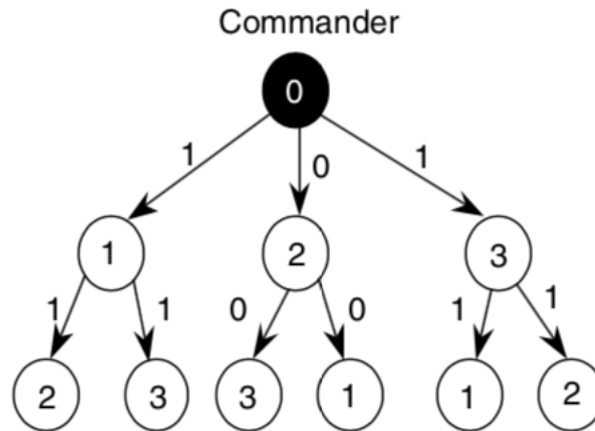
Ex. OM(1)

- 4 Generals

- 1 of them is traitor
- Majority decision function



(a)



(b)

For $f > 1$ (need to adjust the rules – e.g., takes majority of messages even from the same commander)

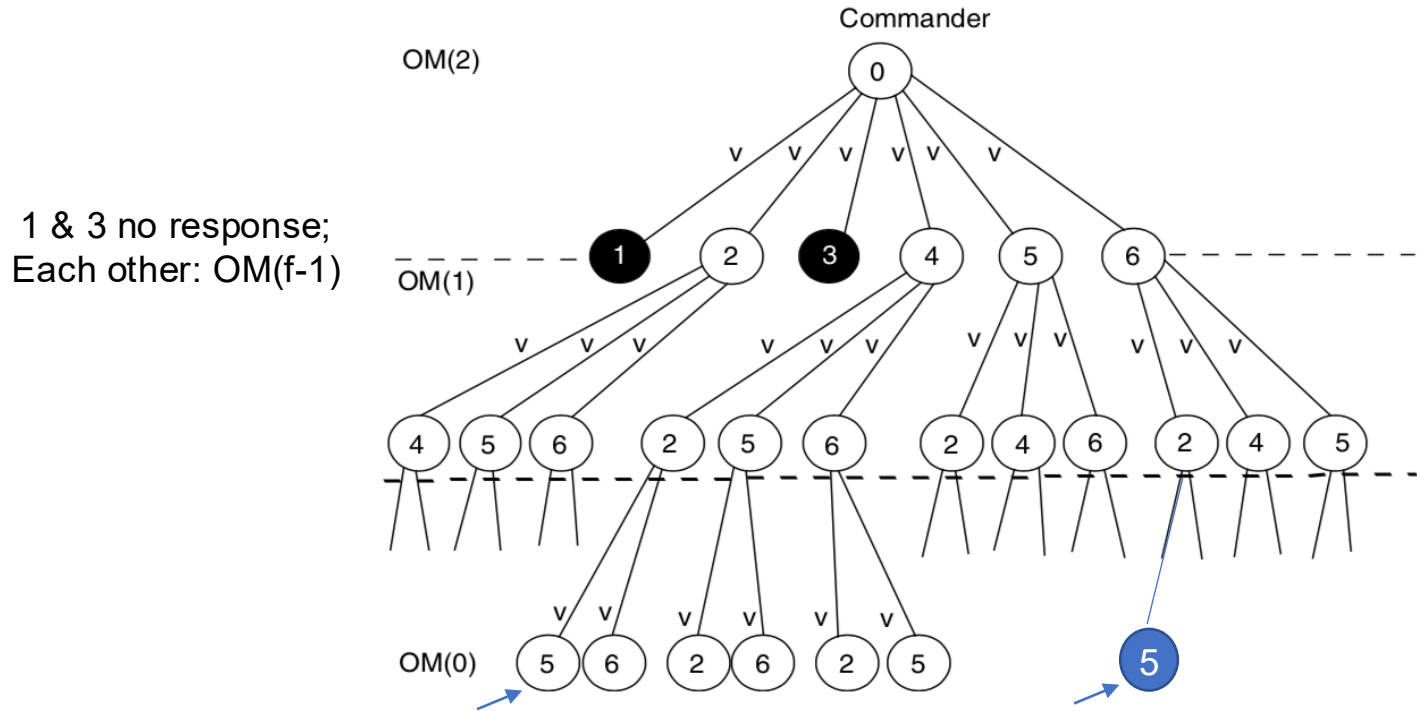


FIGURE 13.6 The OM(2) algorithm with seven generals and two traitors.

PBFT

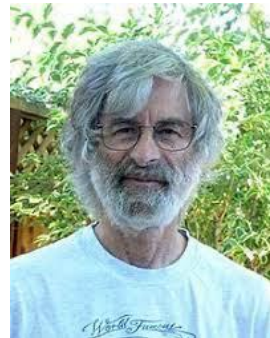
- Setting: Byzantine
- Weakly-sync algorithm
- Use of crypto (message authentication) to solve Byzantine
- Guarantees liveness when weakly sync holds (i.e., the network behaves normally)

Asynchronous	No upper bound on the delay Δ of messages or delay Φ of processors But != message lost	$\Delta/\Phi=(0,\infty)$
Partial Synchrony	There is an unknown upper bound on Δ/Φ	$\Delta/\Phi = (0,X)$
Semi Synchrony	Assume there is known probability distribution on Δ/Φ	$\Delta/\Phi \sim N(x,y)$
Weak Synchrony	Δ/Φ is varying, but it grows at most polynomials in timeout t	$\Delta/\Phi = 2t$
Full Synchrony	There is a known upper bound on Δ/Φ	$\Delta/\Phi = (0,10ms)$



Paxos

- Setting: no Byzantine
- Each instance: one of proposals (multiple concurrent command) reaches consensus
- Next command? Starts a new Paxos instance
- No 100% guarantees of liveness (though guarantees liveness with high probability)
- That is, it works correctly under partial synchrony
 - When network is normal
 - It is safe
 - It provides liveness
 - It terminates
 - When network is abnormal
 - It is **still** safe
 - It may give you liveness and terminate after the network is back to normal
- The protocol attempts to make progress even
 - during periods when some (bounded number) of nodes are unresponsive
- There is also a mechanism to support **configuration changes**
 - drop a permanently failed replica or to add a new replica



Paxos

- First by Lynch et al.

- [But no proof]

- Lamport's Part-Time Parliament Paper [Protocol + Proof]

- *"Inspired by my success at popularizing the consensus **problem** by describing it with Byzantine generals, I decided to cast the **algorithm** in terms of a parliament on an ancient Greek island."*
- *"I submitted the paper to TOCS in 1990. All three referees said that the paper was mildly interesting, **though not very important**, but that all the Paxos stuff had to be removed. I was quite annoyed at how humorless everyone working in the field seemed to be, so I did nothing with the paper."*
- Real systems began using Paxos, follow-up papers were appearing, so TOCS reconsidered the same paper and got accepted in 1998 [8 years!]

The Part-Time Parliament

LESLIE LAMPORT

Digital Equipment Corporation

Recent archaeological discoveries on the island of Paxos reveal that the parliament functioned despite the peripatetic propensity of its part-time legislators. The legislators maintained consistent copies of the parliamentary record, despite their frequent forays from the chamber and the forgetfulness of their messengers. The Paxos parliament's protocol provides a new way of implementing the state-machine approach to the design of distributed systems.

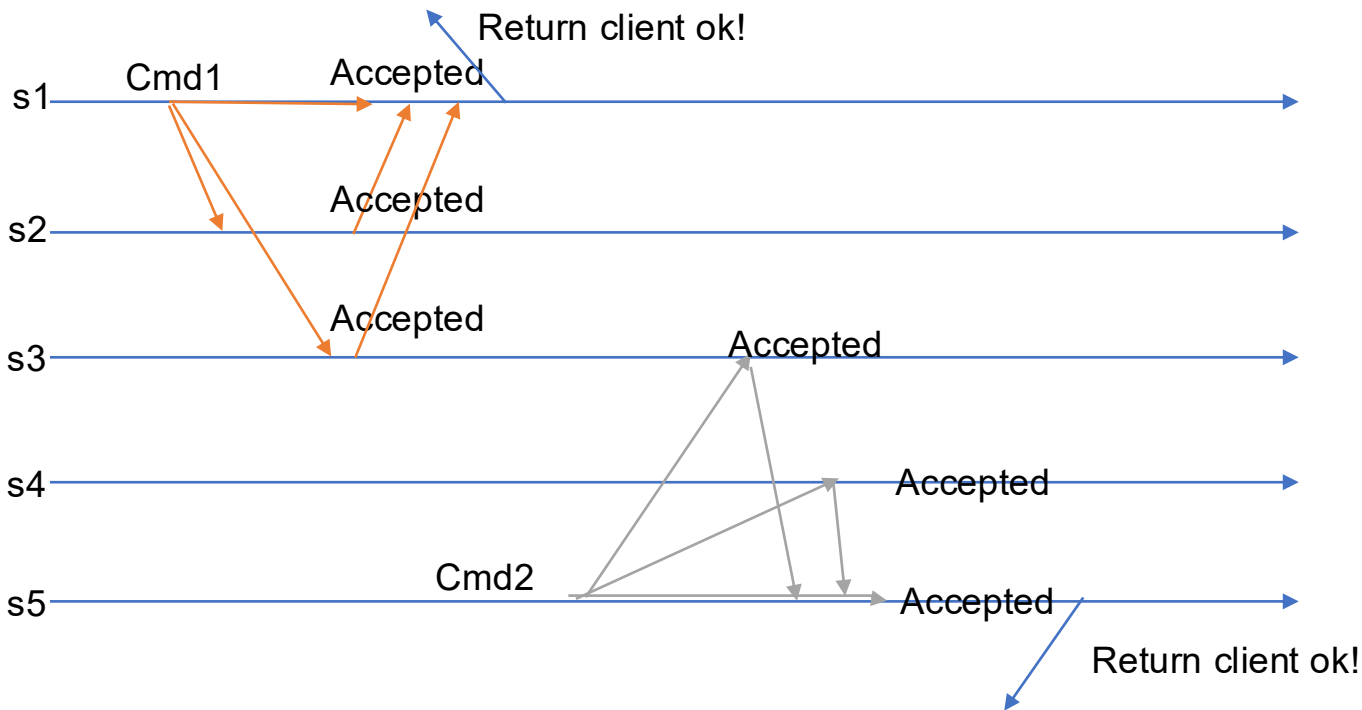
Paxos

- Let's consider this naive protocol:
 - Phase 1: Send a command to **all** replicas
 - Phase 2: Return “commit” to client if **all replicas** said ok
 - What's the problem?



What about this 1-phase “majority” protocol?

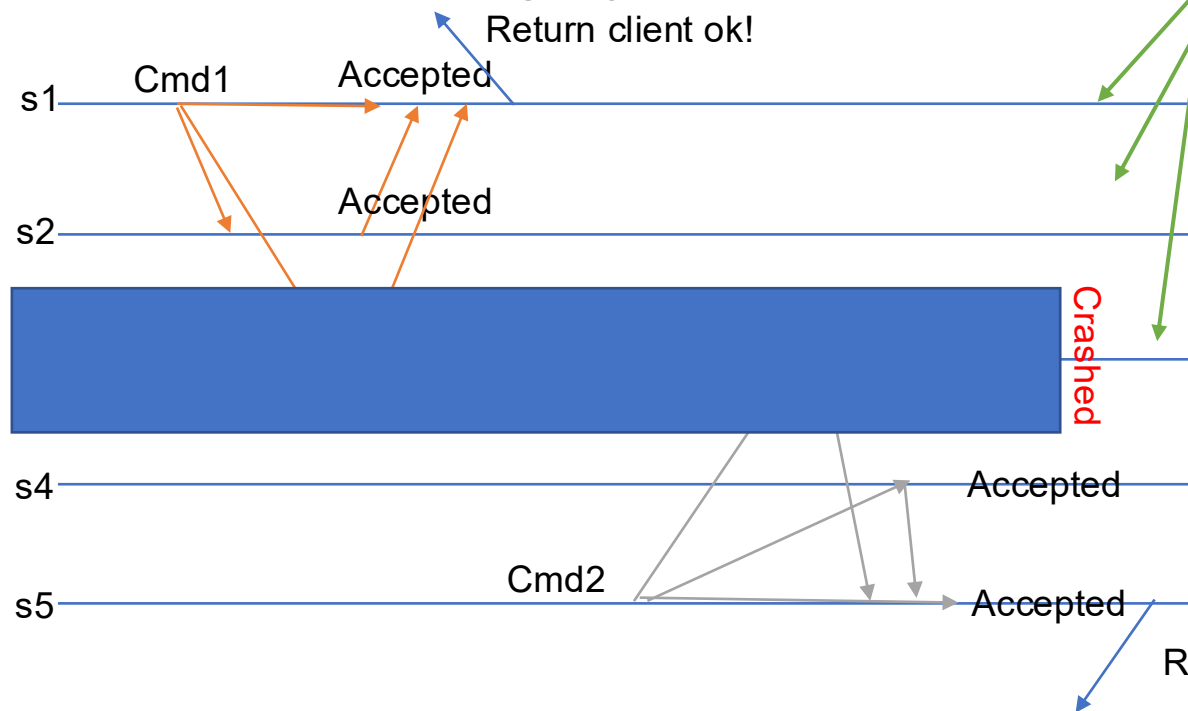
- Accept and Read with the majority.



*2 successful instances here

What about this 1-phase “majority” protocol?

- Accept and Read with the majority.



Client read from majority

Case 1:

s1's log: [Cmd1 (committed)]
s2's log: [Cmd1 (accepted)]
s4's log: [Cmd2 (accepted)]
Return based on: [Cmd1 (committed)]

Case 2:

s2's log: [Cmd1 (accepted)]
s5's log: [Cmd2 (committed)]
s4's log: [Cmd2 (accepted)]
Return based on: [Cmd2 (committed)]

Not linearizable!

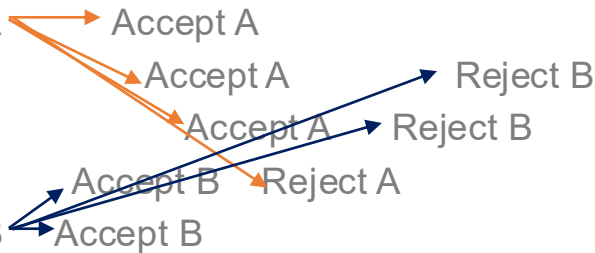
Case 3:

s1's log: [Cmd1 (committed)]
s4's log: [Cmd2 (accepted)]
s5's log: [Cmd2 (committed)]
Return based on: [Cmd1 (committed),
Cmd2 (committed)]

*2 successful instances here

Fixing

- By having 2 phases → Paxos
- By having 1 phase + writing/reading to $> \frac{1}{2}$ replicas (not $\geq \frac{1}{2}$) → Fast-Paxos
 - But a higher number of quorum → less fault-tolerant
 - And when having concurrent commands but none of them can get a quorum?
 - E.g., within the same instance having concurrent commands A and B
 - S1: cmd A → Accept A
 - S2: → Accept A
 - S3: → Accept A → Reject B
 - S4: → Accept B → Reject A
 - S5: cmd B → Accept B
 - Restart the process and fall back to 2-phase Paxos



Paxos

- Similar to Byzantine Generals, a node can wear one of the 3 hats at anytime:
 - Proposer: propose a command, e.g., S1: put(x=17); S2: put(x=18)
 - Voter/Acceptor:
 - vote for a proposal #, e.g., S3 accepts S1's proposal #, turn down S2's #
 - replicas that are part of the quorum (i.e., the majority) in the replication phase
 - Learner: replicas that learn the decisions

One challenge of General Consensus

- 3 proposals (v1, v2, v3), 3 acceptors
 - Can't reach majority that easy
 - s1 votes for v3
 - s2 votes for v2
 - s3 votes for v1
 - No proposal gets enough majority and all get rejected

Paxos

- Before we begin, remember
 1. The parliament story is just a “bad” analogy (that’s why reviewers didn’t like that much and people didn’t understand it)
 2. There are no bad guys
 3. Everyone just wants to get the consensus ASAP indeed (efficiency)
- Phase 1: [leader election] //multiple proposals? After the new leader L is elected, L will get the say
 - A) Prepare
 - B) Promise
- Phase 2: [atomic total-order broadcast] //replication
 - A) Accept
 - B) Accepted

- Proposer

1a) Prepare Phase

- Whoever interested in proposing a value sends out a <Prepare> message, with a new global unique proposal number **N** (e.g., proposer-ip.proposal-ID) to all voters in attempt to be a leader; //only becoming a leader can propose new value

- Voter

- **IF** **N** > **m** // "hi candidate" [where **m** is max of proposal # that I have ever received]
 - **If** I have already **Accepted** (Phase 2b) a proposal value **u**
 - I reply with a **Promise**(**m**, **u**) message // "Ok, I vote for you; but I previously accepted **u**; if you eventually win, you as the new leader better help to finish u first"
 - **Else** // "I promise that I will ignore any received/future proposal number that is <N"
 - I reply with a **Promise**(N, "any value is fine") message // *may Promise >=1 candidate here
- **ELSE**: //ignore N's proposal; no reply here to save communication cost

1b) Promise Phase

- Proposer

- If my proposal **N** has received "a majority of "Promise", then start Phase 2 // "I am the new leader"
 - **If** all received promises say 'any value is fine', then I can decide the value v
 - Send **Accept**(N, v) to all
 - **Else** probably the previous leader is dead and let me finish the unfinished value v
 - Send **Accept**(N, **u'**) to all/a majority of voters
 - where **u'** is value from the one with the highest proposal number **m_{max}** among the set of all replied Promise messages: {(**m₁**, **u₁**), ..., (**m_{max}**, **u'**), ..., (N, 'any'), ...}

2a) Accept Phase

● Acceptor

- If I receive an Accept (N, v) message from leader
 - If $N \geq \text{max proposal \# that I have ever promised}$
 - Send **Accepted**(N, v) ack message to the Proposer**
 - Else /*
 - Ignore this Accept Message

2b) Accepted
Phase

● Proposer (Leader)

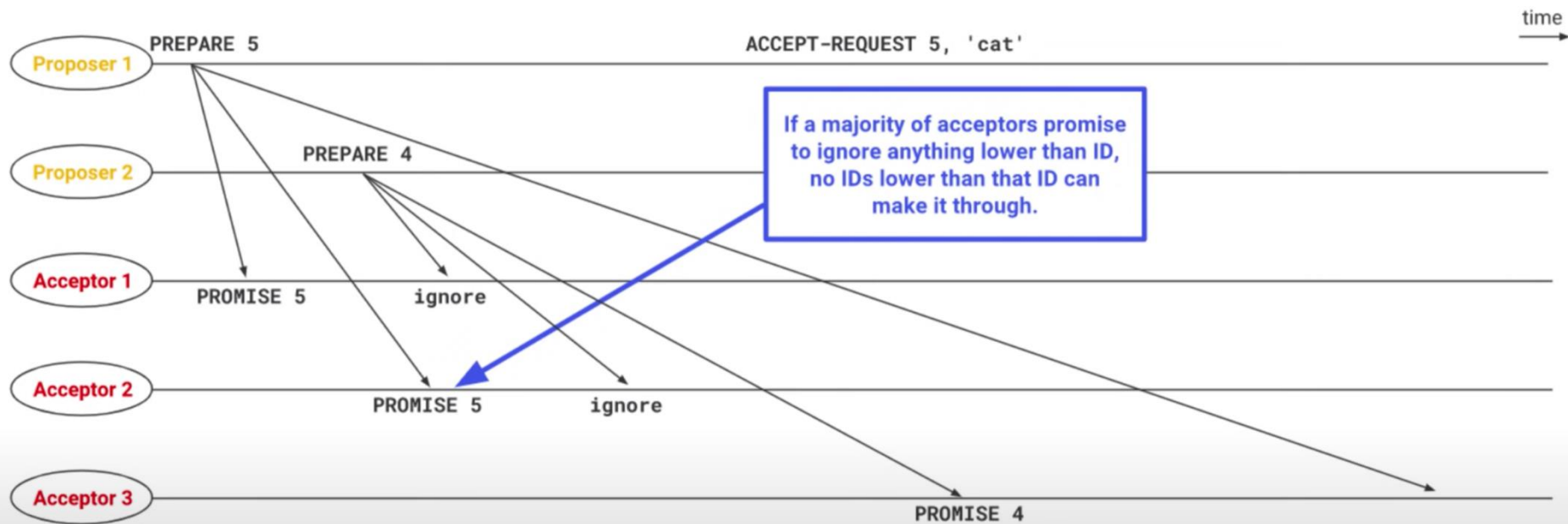
- If the leader receives “Accepted” message from **the majority of voters** with the same proposal number N
 - Consensus is reached and the value is regarded as ‘committed’
 - Notify all (learners) about Accepted v**

** To improve latency, many implementations change it to:

- the voter also sends Accepted(v) to all learners straight (in addition to ack back to the proposer)
- and let each learner counts the majority locally itself (instead of letting the proposer to count the majority and notify the learners**)

→ Save one round-trip

Detailed Review: Majority of Promises



The hardness of consensus algorithm (implementation)

- Is not the happy path
- Is its completeness – the numerous corner cases that we need to handle:
 - Leader / voter dies
 - Leader / voter slow
 - Not enough voters to form a quorum (i.e., $>$ majority vote)
 - Dead leader suddenly resurfaces
 - Split votes
 - ...
 - New voter joins (cluster membership changes)
 - Network partitioning
 - Think of the complexity of Git but need to fully automated (e.g., auto resolve conflicts)!

Caveat

- The FLP result does **not** state that consensus can never be reached:
 - It merely states that under the **asynchronous model**, no **deterministic** algorithm can **always** reach consensus in **bounded** time. **In practice it is highly unlikely to occur.**
 - **Worst case**
- Question: what is the consequence of using Paxos (partially-async algo) in practice (async world)?
 - Answer: Don't complain to Lamport if you occasionally find Paxos hang (i.e., no termination)
 - In practice, if runs too long, simply restart and retry ;)

Summary

- Multi-Paxos (Practical version) → Raft
 - **Stable-leader**
 - Only elect a new leader when the current leader dies
 - **Replicas agree on the log[position] for this command x**
- Agreement problem
 - Usually refers to the 2nd part of Paxos/Raft
 - Single source leader proposes an initial value
 - Make sure everybody else agrees
- Consensus problem
 - Everybody has an initial value to propose and consensus choose one
- Hardness is equivalent to one another